
When Adversarial Prediction Still Compresses: Tokenization and Adversarial Robustness in LLM-Based Lossless Compression

Anonymous Author(s)

Affiliation

Address

email

1 Much of the data stored in real-world business systems is text. It has been shown that Large language
2 models (LLMs) can act as powerful lossless compressors for text data especially: their next-token
3 probabilities coupled with entropy coding can reduce data significantly below its original size and
4 LLM-based prediction has been shown to outperform traditional compression schemes such as
5 FSST and Brotli by up to $2\times$ on real-world data. Making this advantage practical though requires
6 addressing inference cost, throughput, decompression, robustness, and economics; among these
7 concerns, robustness remains understudied. We evaluate an LLM-based lossless compressor as a
8 two-layer system of (1) tokenization and (2) prediction. We challenge/stress-test this system on
9 adversarial inputs that approximately maximize prediction or compression cost mimicking benign
10 data-model misalignment and malicious inputs specifically designed to expand the encoded stream.
11 Crucially, our analysis separates tokenization from LLM-prediction and shows that even severe
12 prediction failure on the part of the LLM need not cause compression failure: unlikely tokens can
13 decode to many source bytes and thereby buffer high prediction cost. Our byte-normalized adversary
14 removes this protection: although MinProb raises prediction cost by $5.68\times$ while the complete stream
15 remains at 36.2% of raw size, MaxSurprisal/Byte expands it to 144.3%.

16 1 Who Is Actually Doing the Heavy Lifting in LLM-Based Compression?

17 Strings are a core data type in production databases: in an Amazon Redshift fleet study, variable-
18 length strings account for 52.1% of stored columns and 53.8% of predicate columns [1]. Production
19 text data, however, is heterogeneous and may not match an LLM’s training distribution, which
20 is something general-purpose systems must therefore be able to cope with. As data accumulates,
21 compression becomes especially important for cold storage, where space savings can be weighed
22 favorably against access latency and encoding cost [2]. Yet favorable storage economics are not
23 enough: a practical LLM codec must also meet broader systems requirement. Such a codec
24 must preserve queryability, for example, supporting equality checks or joins without full decoding,
25 while remaining robust to arbitrary inputs and degrading gracefully when model predictions are poor.

26 To this end, we study a layered architecture designed around the separation from tokenization and
27 predictive compression. The tokenizer first maps strings to stable token IDs that act as a global
28 dictionary and support operations over encoded data without model inference [3]; these IDs form
29 the lossless, queryable representation. For cold storage or transport, an LLM predicts the token
30 IDs and an arithmetic coder compresses them. Because tokenization already constitutes a lossless
31 pre-compression, this predictive layer is optional. When it is used tough, arithmetic coding assigns
32 shorter descriptions to more probable tokens [4, 5].

33 Prior LLM codecs do already combine next-token probabilities with arithmetic coding or other
34 lossless coding schemes [6]. Prediction, however, is only once source of compression. Tokenization
35 itself can also act as lossless pre-compression [7]. The popular tokenization strategy BPE (Byte-Pair
36 Encoding) even originated as a compression algorithm [8] in the first place, and its effectiveness
37 has been linked to shorter token sequences [9]. From a systems perspective however, these gains

ultimately only matter insofar as the complete codec improves on conventional alternatives such as Brotli [10] and FSST [11].

What remains unclear is how much of an LLM codec’s behavior comes from tokenization versus prediction. Delétang et al. [7] compare ASCII and BPE [12] on natural text, but do not isolate these effects under inputs targeting realized end-to-end size. This distinction matters because prediction cost is measured in bits/token, whereas compression depends on bits/source byte: a poor probability model can still compress when tokens represent many bytes. We therefore isolate tokenization and prediction under adversarial inputs to determine when prediction failure becomes compression failure and when tokenization absorbs the cost.

Contributions. We make two contributions: (1) we factor end-to-end compression into tokenizer fertility and predictive coding under natural, random, and adversarial inputs. This decomposition reveals that prediction failure need not imply compression failure: on text8, Qwen2.5-0.5B with arithmetic coding produces a complete stream at 17.6% of raw size, and even when the MinProb adversary raises prediction cost by $5.68\times$, the stream remains compressive at 36.2%. (2) we formalize token- and byte-level adversarial objectives with realized-code accounting, showing when tokenization absorbs predictive surprise and when that protection against size expansion breaks down. Directly maximizing decoded-byte-normalized surprisal removes this buffer and expands the complete stream to 144.3% of raw size.

2 Compression Objectives and How to Construct Adversarial Inputs

A token can be extremely unlikely under the language model while decoding to few source bytes. We formalize the central distinction between prediction difficulty and end-to-end compression failure. We use one notation throughout: let $x = (x_1, \dots, x_N)$ be the generated token sequence, let $h_t = x_{<t}$ be its token history, let $D = \text{decode}$ denote the decoder, and let $p_\theta(v \mid h)$ be the language model probability of token v after history h . Let $|\text{decode}(x)|_{\text{UTF8}}$ denote the number of source bytes obtained after decoding the token sequence x . For example, if $\text{decode}(x) = \text{hello}$, then $|\text{decode}(x)|_{\text{UTF8}} = 5$ bytes. Let $B(x)$ be shorthand for the decoded source size in bytes: $B(x) = |\text{decode}(x)|_{\text{UTF8}}$.

The token-level objective, MinProb, selects the least-probable admissible continuation $v_{\text{tok}}^* \in G(h)$:

$$v_{\text{tok}}^* = \arg \min_{v \in G(h)} p_\theta(v \mid h) = \arg \max_{v \in G(h)} \{-\log_2 p_\theta(v \mid h)\}. \quad (1)$$

This is the LLM-centric token-level adversarial objective, which maximizes model surprisal and ideal coded bits/token equivalently, but not necessarily bits/source byte. To account for variable-length tokenization, define the incremental decoded byte length contributed by the token v ,

$$\Delta b(v; h) = |\text{decode}(h \parallel v)|_{\text{UTF8}} - |\text{decode}(h)|_{\text{UTF8}}. \quad (2)$$

The byte-aware MaxSurprisal/Byte objective instead selects v_{byte}^* :

$$v_{\text{byte}}^* = \arg \max_{v \in G} \frac{-\log_2 p_\theta(v \mid h)}{\Delta b(v; h)} \quad [\text{bits/byte}]. \quad (3)$$

Unlike Eq. 1, it directly targets the ideal entropy-coding cost in bits/newly decoded source byte. For a complete sequence x , the cumulative ideal model cost is the sequence NLL $L_\theta(x)$

$$L_\theta(x) = \sum_{t=1}^N \ell_\theta(x_t \mid x_{<t}) = - \sum_{t=1}^N \log_2 p_\theta(x_t \mid x_{<t}) \quad [\text{bits}], \quad (4)$$

where any fixed initial context is implicit in $x_{<1}$. For serialized output size $S_{\text{codec}}(x) = |\text{SerializeCodec}(x)|$ in bytes, the ideal and realized source-normalized size ratios are

$$R_{\text{ideal}}(x) = 100 \frac{L_\theta(x)}{8B(x)} [\%] \quad R_{\text{real}}(x) = 100 \frac{S_{\text{codec}}(x)}{B(x)} [\%]. \quad (5)$$

Values below 100% denote compression; R_{real} additionally captures arithmetic-coder, metadata, and framing effects.

76 2.1 Prediction Loss Is Not Compression Loss

77 We define tokenizer fertility as the average decoded source-byte count:

$$f(x) = \frac{|\text{decode}(x)|_{\text{UTF8}}}{N} \quad [\text{bytes/token}]., \quad (6)$$

78 Using the sequence NLL $L_\theta(x)$, the ideal predictive rate is

$$R_{\text{ideal}}(x) = 100 \frac{L_\theta(x)}{8|\text{decode}(x)|_{\text{UTF8}}} = \underbrace{\frac{L_\theta(x)}{N}}_{\text{bits/token}} \underbrace{\frac{N}{|\text{decode}(x)|_{\text{UTF8}}}}_{\text{tokens/byte}} \frac{100}{8}. \quad (7)$$

79 Thus high bits/token can be offset by high bytes/token. Tokenization can buffer severe prediction
80 failure when adversarial tokens represent many source bytes. Looking at 7, hence, there is a
81 compression boundary for R_{ideal} that occurs at $\frac{L_\theta(x)}{N} < 8 \frac{|\text{decode}(x)|_{\text{UTF8}}}{N} = 8f(x)$. A token
82 representing f source bytes can tolerate up to $8f$ bits of surprisal before the predictive layer expands
83 the source.

84 2.2 Token- and Byte-Level Attacks

85 **Threat Model.** We assume white-box access to the compression pipeline: the adversary can
86 inspect the tokenizer and model logits and constructs length- N adversarial inputs by autoregressive
87 search over a candidate alphabet G . To ensure that generated token sequences survive decoding and
88 re-tokenization, every prefix must satisfy $T(\text{decode}(x_{1:t})) = x_{1:t}$; see Appendix 5.3.

89 **Adversarial Input Generation.** We benchmark natural Wikipedia text and random strings against
90 two adversarial generation policies: *Minimum probability* (v_{tok}^*) greedily maximizes Eq. 1, while
91 *maximum surprisal/byte* (v_{byte}^*) greedily maximizes Eq. 3. In addition, we provide a *one-byte UTF-8*
92 *ablation* restricting the candidate alphabet G to canonical tokens representing exactly one decoded
93 byte, thereby removing variation in token fertility. Under this constraint, the multi-byte buffering
94 is removed and the v_{tok}^* and v_{byte}^* objectives induce the same greedy ranking. These inputs model
95 adverse blocks arising from either benign data-model misalignment or malicious users. Among
96 predictive objectives, MaxSurprisal/Byte (v_{byte}^*) captures the relevant failure mode for end-to-end
97 size because token-level loss alone does not determine serialized size.

98 3 Evaluation

99 3.1 Experimental Setup

100 We evaluate Qwen2.5-0.5B [13] on natural text, random text input, full-vocabulary adversarial
101 sequences, and a one-byte UTF-8 ablation. Each input in the attack evaluation contains 10,000 tokens,
102 except for the full-vocabulary MaxSurprisal/Byte (v_{byte}^*) evaluation, which contains 1,024 tokens
103 due to computational constraints.¹ All attacks use the same model, context, and arithmetic-coding
104 configuration, and generated prefixes must round-trip through tokenizer decoding and re-encoding.

105 3.2 Results

106 **Natural-to-Random Shift Removes the Predictive Advantage.** As was to be expected, Qwen+AC
107 is the best performing compressor on natural text (17.6%), all algorithms have acceptable performance.
108 Interestingly, tokenization only is already a strong baseline, better than FSST even - this is explained
109 by the fact that the Qwen tokenizer is trained on text (learned). On random text, tokenization ...
110 whereas prediction still repairs much of the Qwen token IDs baseline’s inefficiency, but under this
111 distribution shift it no longer beats a conventional codec. It is worth noting that random input alone is
112 still not able to break the 100% compression barrier for Qwen+AC.

¹This shorter evaluation does not change the qualitative conclusion; see Appendix 5.4.

Table 1: **Source-normalized predictive coding under increasingly adverse inputs.** Token-level surprisal alone is not predictive of compression failure. ↓ lower is better, ↑ higher is better for compression. † Final 1,024-token evaluation (one run).

Category	Input / condition	Pred. cost ↓ (bits/token)	Fertility ↑ (bytes/token)	Ideal rate ↓ (bits/byte)	AC rate ↓ (bits/byte)
Baselines	Natural text (text8)	5.11	5.59	0.92	1.13
	Random canonical	8.78	1.31	6.71	7.58
Adversaries	MinProb (v_{tok}^*)	29.04	7.09	4.10	2.53 [‡]
	MaxSurprisal/Byte (v_{byte}^*) [†]	15.52	1.47	10.53	9.79
Ablation	One-byte UTF-8	8.33	1.00	8.33	9.57

[‡] The unexpectedly low realized AC rate reflects finite-frequency quantization in the practical arithmetic coder, together with coder-support restriction and renormalization; it does not indicate improved prediction.

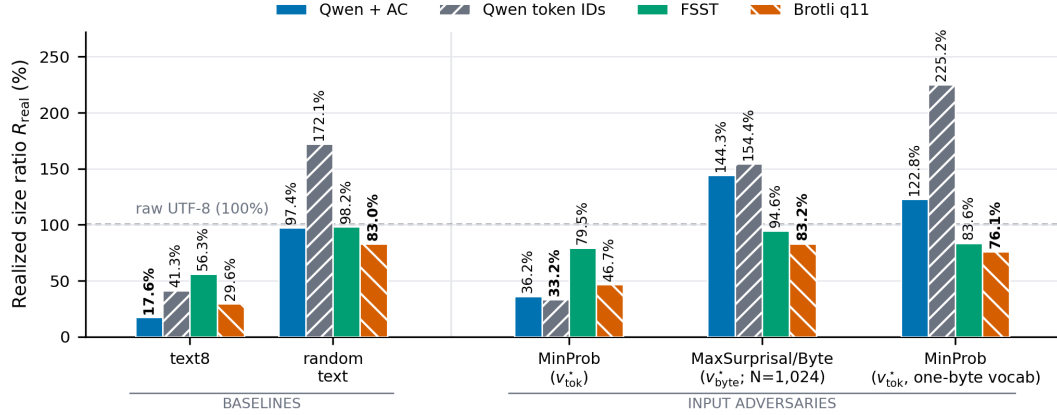


Figure 1: **End-to-end compression across natural vs. adversarial inputs.** Realized size ratio R_{real} for Qwen+AC, Qwen token IDs, FSST [11], and Brotli. Bold values mark the smallest stream for each input. All streams are round-trip-verified. Qwen model weights, tokenizer table are excluded.

Prediction Failure Does Not Imply Compression Failure. The MinProb (v_{tok}^*) attack increases prediction difficulty for the LLM $5.68\times$, yet remains compressive because tokenizer fertility is high at 7.09 source bytes/token. This is because MinProb simultaneously affects tokenization: by selecting unlikely continuations directly in token space, it selects vocabulary items that frequently decode to long, multi-byte inputs, as reflected by the increase in fertility in Table 1 (from 5.59 to 7.09). The Qwen token IDs baseline is slightly smaller than Qwen+AC on the same bytes, confirming that robustness comes from the multi-byte token representation rather than an additional predictive-coding gain. For Qwen token IDs, random input is in fact more difficult than MinProb input, underscoring that tokenization alone provides substantial pre-compression.

Attacking Fertility-Adjusted Cost Causes Expansion. Restricting MinProb to a one-byte token vocabulary is one way of removing the fertility buffer and makes the complete Qwen+AC stream expand to 122.8%. MaxSurprisal/Byte (v_{byte}^*) attacks the same buffer directly over the full vocabulary and likewise causes expansion and is the most effective joint attack on prediction + tokenization. MaxSurprisal/Byte (v_{byte}^*) is the most effective attack on the complete Qwen + AC codec.

4 Implications and Conclusion

LLM-Based Compression Survives Model Errors. LLM compression is an end-to-end systems property, not merely a property of the language model: tokenization, prediction, coding, and serialization jointly determine final size. On natural text (text8), the complete Qwen+AC reaches 17.6% of raw size. MinProb (v_{tok}^*) raises prediction cost to $5.68\times$ its text8 level, yet its payload remains at 31.7% because multi-byte tokens absorb the surprise; the Qwen token IDs baseline is slightly smaller than Qwen+AC on the same MinProb input. Conversely, restricting candidates to one-byte tokens removes the tokenization-buffer and makes the complete stream expand to 122.8%. Compression

135 therefore survives poor prediction because of tokenizer fertility, not necessarily because the model
136 predicts well.

137 **Queryable Token Representation as the Foundation.** Token IDs provide a stable, queryable
138 representation for hot data, while predictive coding can be reserved for sufficiently cold blocks.
139 This separation matters because LLM compression trades storage savings for inference cost and
140 access latency, making access frequency and retention time central to whether it is worthwhile [2].
141 A conventional codec such as Brotli can serve as a fallback, ensuring that predictive compression
142 remains an optional optimization rather than a required representation.

143 References

- 144 [1] Alexander van Renen, Dominik Horn, Pascal Pfeil, Kapil Vaidya, Wenjian Dong, Murali
145 Narayanaswamy, Zhengchun Liu, Gaurav Saxena, Andreas Kipf, and Tim Kraska. Why TPC is
146 not enough: An analysis of the amazon redshift fleet. *Proceedings of the VLDB Endowment*, 17
147 (11):3694–3706, 2024. doi: 10.14778/3681954.3682031.
- 148 [2] Andreas Kipf et al. Waiting to decompress: The economics of llm-based compression. In *CIDR*,
149 2026.
- 150 [3] Tobias Schmidt et al. Accelerating string-heavy queries with LLM token tables. *PVLDB*, 19,
151 2026.
- 152 [4] Ian H. Witten et al. Arithmetic coding for data compression. *Commun. ACM*, 30(6):520–540,
153 June 1987. doi: 10.1145/214762.214771.
- 154 [5] Christian Steinruecken. *Lossless Data Compression*. PhD thesis, University of Cambridge,
155 Cambridge, UK, 2015.
- 156 [6] Chandra Shekhara Kaushik Valmeekam et al. LLMZip: Lossless text compression using large
157 language models. *arXiv preprint arXiv:2306.04050*, 2023.
- 158 [7] Grégoire Delétang et al. Language modeling is compression. In *ICLR*, 2024.
- 159 [8] Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.
- 160 [9] Matthias Gallé. Investigating the effectiveness of BPE: The power of shorter sequences. In
161 *EMNLP-IJCNLP*, pages 1375–1381, Hong Kong, China, 2019. Association for Computational
162 Linguistics. doi: 10.18653/v1/D19-1141.
- 163 [10] Jyrki Alakuijala et al. Brotli compressed data format. RFC 7932, Internet Engineering Task
164 Force, July 2016. URL <https://www.rfc-editor.org/info/rfc7932>.
- 165 [11] Peter Boncz et al. FSST: Fast random access string compression. *Proceedings of the VLDB*
166 *Endowment*, 13(11):2649–2661, 2020. doi: 10.14778/3407790.3407851.
- 167 [12] Rico Sennrich et al. Neural machine translation of rare words with subword units. In *ACL*,
168 pages 1715–1725, Berlin, Germany, 2016. Association for Computational Linguistics. doi:
169 10.18653/v1/P16-1162.
- 170 [13] Qwen et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. doi: 10.48550/
171 arXiv.2412.15115. URL <https://arxiv.org/abs/2412.15115>.

5 Appendix

5.1 Metric Definitions

For a token sequence x of length N , let $B(x) = |\text{decode}(x)|_{\text{UTF8}}$ be its decoded UTF-8 size in bytes, $L_\theta(x)$ its model negative log-likelihood in bits, and $S_{\text{AC}}(x)$ its arithmetic-coded payload in bits. We report prediction cost and tokenizer fertility per token, and predictive coding cost per source byte:

$$\text{Prediction cost} = \frac{L_\theta(x)}{N} \quad [\text{bits/token}], \quad (8)$$

$$\text{Tokenizer fertility} = \frac{B(x)}{N} \quad [\text{bytes/token}], \quad (9)$$

$$\text{Ideal predictive rate} = \frac{L_\theta(x)}{B(x)} \quad [\text{bits/byte}], \quad (10)$$

$$\text{Realized AC rate} = \frac{S_{\text{AC}}(x)}{B(x)} \quad [\text{bits/byte}]. \quad (11)$$

The ideal rate factorizes as $L_\theta(x)/B(x) = [L_\theta(x)/N]/[B(x)/N]$: high prediction cost can therefore be offset when each token represents many source bytes. To compare encoded sizes directly with raw UTF-8, we report

$$R_{\text{AC}}(x) = 100 \frac{S_{\text{AC}}(x)}{8B(x)}, \quad (12)$$

$$R_{\text{real}}(x) = 100 \frac{|\text{SerializeCodec}(x)|}{B(x)}. \quad (13)$$

Here R_{AC} covers only the arithmetic payload, whereas R_{real} covers the complete serialized stream in bytes, including its dictionary, metadata, and framing. Both are percentages of the raw source size: values below 100% denote compression and values above 100% denote expansion.

5.2 How We Generate the Adversarial Inputs

Each attack builds its input one token at a time. At every step, it (1) asks the model to score the next token, (2) ranks the candidate tokens according to the attack objective, and (3) tries candidates in that order until it finds one that remains unchanged after decoding and retokenization. The accepted token is then appended and the process repeats.

Candidate pool. We begin with every token ID shared by the tokenizer and model, then remove all IDs listed in `tokenizer.all_special_ids`. This excludes EOS, padding, chat markers, and other tokenizer-declared control tokens. The resulting pool contains 151,643 IDs for Qwen2.5-0.5B. The notation $G(h)$ in the main paper means the members of this pool that have a finite model score and pass the exact round-trip check in Algorithm 3 after prefix h . MaxSurprisal/Byte also requires the token to add at least one decoded byte. Admissibility depends only on the current prefix and proposed next token; there is no lookahead.

Algorithm 1: MinProb

Choose the least likely valid token

Input: a nonempty seed prefix h and target length N .

1. Ask the model for the next-token logits.
2. Remove candidates with non-finite logits. Sort the rest from lowest logit to highest logit; for equal logits, place the lower token ID first.
3. In that order, apply Algorithm 3. Append the first candidate that passes.
4. Repeat until the complete sequence, including the seed, has N tokens.

Output: the completed token sequence h .

Algorithm 2: MaxSurprisal/Byte *Choose the most surprising valid token per added byte*

Input: a nonempty seed prefix h and target length N .

1. Ask the model for the next-token logits and compute float32 log probabilities over the full tokenizer vocabulary.
2. For every finite candidate v , decode the complete prefixes h and $h\|v$ and measure $d_v = |D(h\|v)|_{\text{UTF8}} - |D(h)|_{\text{UTF8}}$. Remove candidates for which $d_v \leq 0$.
3. Rank the remaining candidates by decreasing $-\log_2 p(v | h)/d_v$; for equal scores, place the lower token ID first.
4. In that order, apply Algorithm 3. Append the first candidate that passes.
5. Repeat until the complete sequence, including the seed, has N tokens.

Output: the completed token sequence h .

Algorithm 3: Exact round-trip check *Accept only tokenizations that a source string can reproduce*

Input: the accepted prefix h and proposed token v .

1. Decode $h\|v$ with special-token skipping and tokenizer cleanup disabled.
2. Retokenize the decoded text with `add_special_tokens=False`.
3. Accept v exactly when the resulting IDs equal $h\|v$; otherwise reject it and test the next ranked candidate.

Output: accept or reject.

What is evaluated at each step. Neither attack round-trip tests all 151,643 candidates. Both rank the candidates first and perform Algorithm 3 only until one passes. MinProb tests exactly the leading part of the ranked list, ending with the first candidate that passes. MaxSurprisal/Byte does, however, decode the prefix with every candidate to obtain all byte increments before ranking. Measuring the complete prefix rather than decoding a token by itself captures whitespace merging and other tokenizer-boundary effects. Candidates that add zero or fewer bytes are discarded by MaxSurprisal/Byte; MinProb has no byte-length rule. An invalid UTF-8 byte fragment may decode to a replacement character, but it is accepted only if retokenizing that decoded text recovers the exact proposed IDs.

Faster round-trip check with an exact fallback. Algorithm 3 states the acceptance rule used by the experiments. The implementation avoids repeatedly retokenizing a long prefix when it can prove that an earlier part cannot change. It caches raw bytes, decoded text, token and character byte offsets, and pre-tokenizer pieces for accepted prefixes. It may retokenize only the suffix beginning at the penultimate pre-tokenizer piece when all of the following hold: the suffix begins at a token boundary, is at most 512 bytes long, begins with ASCII, already retokenizes to its recorded IDs, and its first pre-tokenizer piece does not change after adding the candidate. If any condition fails, including for an invalid UTF-8 fragment, the code performs the full-prefix check in Algorithm 3. The shortcut changes runtime but never the acceptance decision.

Reproduction settings.

Setting	Value used in the reported experiments
Seed prefixes	MinProb: The = [785], A = [32], and In = [641]. MaxSurprisal/Byte: A = [32]. Total length includes the seed.
One-byte ablation	Scan the vocabulary once; retain v exactly when $[v]$ decodes to one ASCII code point with value 0–127 and that character retokenizes to $[v]$. Qwen provides exactly 128 such IDs.
Model scores	Pretrained model without weight quantization, loaded with the framework automatic dtype. Logits are converted to float32 for ranking and logsoftmax. Arithmetic-coder frequency quantization happens only after generation and cannot affect the selected tokens.
Context	KV caching is enabled. At 1,000 context tokens, retain the latest 100 and rebuild the cache. Byte measurement and round-trip validation still use the complete generated prefix.
Length and cost	MinProb: 10,000 tokens for each of three seeds. MaxSurprisal/Byte: 1,024 tokens for one seed because it performs 151,643 context-sensitive decodes per step.
Wall time	The archived runs did not record elapsed time, so no reliable number can be reconstructed. The released implementation reports progress per step; future result files should record hardware and wall time.

5.3 Evaluation Protocol

Canonical Inputs. Not every sequence of valid token IDs can arise from tokenizing its own decoded bytes. For example, suppose the vocabulary contains tokens for a, b, and ab. The token sequence [a, b] decodes to ab, but the tokenizer may encode ab as the single token [ab]. Hence, decoding and retokenizing changes the token sequence. Since our attacks select token IDs directly from the model distribution, allowing such sequences would optimize over token histories that cannot occur for the corresponding source input. We therefore require every generated partial sequence to be canonical, i.e., $T(\text{decode}(x_{1:t})) = x_{1:t}$. This ensures that final decoding recovers exactly the same token IDs and bytes.

Model and Coder. We chose Qwen2.5-0.5B because its small size makes full-vocabulary adversarial scoring tractable while retaining a large vocabulary, and allows the experiments to run locally on a CPU without requiring GPU access. We use 151,643 non-special candidates, a 1,000-token context with 100 tokens retained between coding blocks, and KV caching. Arithmetic coding uses a 32-bit state and frequency total 262,144. The version-1 Qwen+AC stream records the seed, model and coder configuration, token and raw-byte counts, arithmetic payload, any required token dictionary, and framing. Model weights and the shared tokenizer table are shared decoder state and excluded from both Qwen representations.

Inputs and Aggregation. The primary evaluation uses 10,000-token sequences for natural text, random canonical input, MinProb (v_{tok}^*), and the one-byte UTF-8 ablation. MaxSurprisal/Byte (v_{byte}^*) is reported at the largest feasible budget of 1,024 tokens and is the final evaluation for that attack. The reported values for the other adversarial conditions are means over three independent canonical seed runs; the MaxSurprisal/Byte (v_{byte}^*) result is a single run.

Round-Trip Validation and Baselines. All generated prefixes satisfy

$$T(D(x_{1:t})) = x_{1:t}, \quad t = 1, \dots, N,$$

and every complete stream is independently decoded back to identical token IDs or UTF-8 bytes, as applicable. FSST is evaluated using the authors’ reference implementation [11]; Brotli uses quality 11; and the Qwen token IDs baseline stores each ID in a fixed-width 18-bit representation with a stream header. Exact software revisions are recorded in the accompanying artifacts.

5.4 Detailed Results and Accounting

Effect of the Shorter MaxSurprisal/Byte (v_{byte}^*) Sequence. The shorter sequence affects the magnitude but not the qualitative conclusion. Under the same replay configuration, the arithmetic-payload

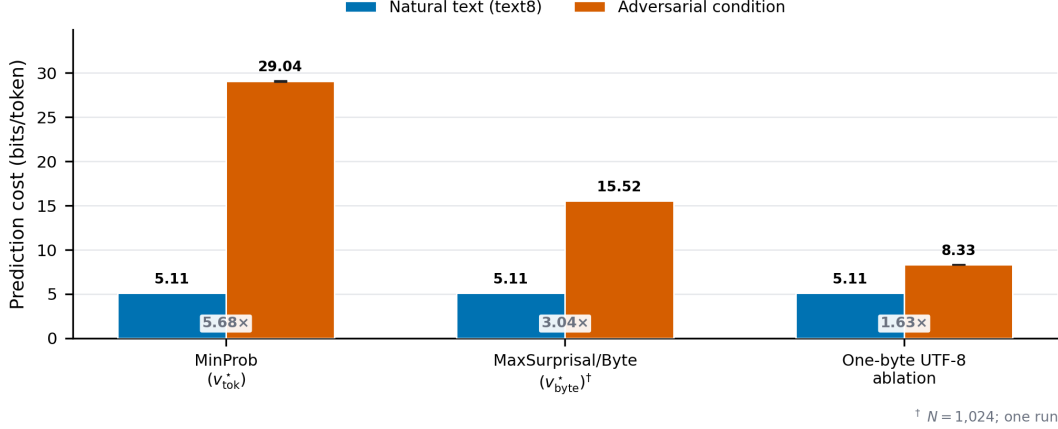


Figure 2: **Adversarial prediction difficulty.** Each condition is compared with the same natural-text prediction cost of 5.11 bits/token. Labels inside the plot report the ratio to text8. Error bars show sample standard deviation across three canonical seeds where available; † marks the final 1,024-token MaxSurprisal/Byte (v_{byte}^*) single run.

ratio is 136.4%, 133.2%, 131.7%, and 122.4% at $N = 32, 64, 128$, and 1,024, respectively; even the longest evaluated prefix remains larger than raw UTF-8. The complete 1,024-token Qwen+AC stream is 144.3% of its 1,508 source bytes. Moreover, all bars in the MaxSurprisal/Byte group use those same source bytes, so their comparison is size matched. Fixed codec overhead is relatively larger on short inputs, which disadvantages rather than favors the conventional codecs.

Codec Comparison. Figure 1 and Table 3 compare codecs on identical source bytes. The four non-Max groups contain 9,996–13,087 bytes; the 1,024-token MaxSurprisal/Byte output contains 1,508 bytes. All plotted results are complete, round-trip-verified streams.

Stream Accounting. The auxiliary comparison reports both the Qwen+AC payload and complete stream, whose difference is 331–334 bytes across the four non-Max serialized inputs. For text8, this overhead raises the size ratio from a 14.3% arithmetic-coder payload to a 17.6% complete stream. The complete size is $S_{\text{total}} = S_{\text{payload}} + S_{\text{dict}} + S_{\text{frame}}$. Finite frequencies clip extremely small probabilities, so realized arithmetic length can be below unquantized NLL for MinProb; we report this as quantization, not improved prediction.

5.5 Arithmetic-Coder Quantization Sensitivity

The coder cannot consume arbitrary real-valued probabilities. For a coding alphabet of size K and frequency total T , it constructs

$$f_i = 1 + \lfloor (T - K)p_i \rfloor + r_i, \quad q_i = f_i/T,$$

where $r_i \in \{0, 1\}$ distributes the counts left after flooring in descending model-probability order. The leading one makes every token decodable, but also imposes $T \geq K$ and a probability floor $p_{\min} = 1/T$. Thus 2^{14} is infeasible for the full $K = 151,665$ coder alphabet. The coder alphabet includes the 22 tokenizer IDs excluded from attack selection, hence 151,665 rather than 151,643. Our reported setting, $T = 2^{18}$, leaves only $T - K = 110,479$ counts to represent differences after reserving one count per token. We define a selected token as *clipped* when it receives only that mandatory count.

Table 2 replays the fixed adversarial token sequences at $T \in \{2^{18}, 2^{22}, 2^{26}, 2^{30}\}$; 2^{30} is the largest power of two admitted by the unchanged 32-bit coder. The model, token sequences, decoded bytes, context policy, and arithmetic-coder state size are held fixed. We distinguish two unquantized references. The model ideal rate in Table 1 normalizes over the model’s 151,936-logit padded output support. The lossless stream can emit only the K tokenizer IDs, so the coder first restricts to that support and renormalizes. For MinProb this support restriction reduces the ideal rate from 4.10 to 3.79 bits/byte; integer-frequency quantization then reduces it to 2.534 bits/byte at $T = 2^{18}$. This decomposition prevents the whole model-ideal-AC gap from being attributed to integer rounding.

At $T = 2^{18}$, 99.0% of selected MinProb tokens are clipped, and the quantized ideal and realized AC rates agree to the reported precision. Raising precision monotonically moves both toward the support-restricted ideal: at the maximum $T = 2^{30}$, clipping falls to 13.8% and realized AC rises to 3.735 bits/byte (46.7% of raw UTF-8), compared with 2.534 bits/byte (31.7%) at 2^{18} . Quantization therefore materially affects the numerical MinProb rate, but does not overturn the tokenization result: even the maximum-precision payload remains less than half of raw size because MinProb has high fertility. For MaxSurprisal/Byte, realized AC changes from 9.790 to 10.481 bits/byte (122.4% to 131.0% of raw); it remains decisively expansive. Across every row, realized AC differs from quantized ideal by at most 0.002 bits/byte. The reported surprisal error is $\Delta s = -\log_2 q_i + \log_2 p_i$ after restricting p_i to coder support; Table 2 reports its per-token mean and maximum absolute value.

Table 2: Arithmetic-coder quantization sensitivity. Fixed token sequences are replayed while only the integer frequency total T changes. Model ideal uses the model’s full output support; support ideal renormalizes over the $K = 151,665$ token IDs available to the lossless coder. Quantized ideal uses the exact integer frequencies. MinProb values are means over three seeds; MaxSurprisal/Byte is the final $N = 1,024$ run. A token is clipped when its selected frequency is the mandatory count one. $\Delta s = s_{\text{quant}} - s_{\text{support}}$ is in bits/token; the last column gives mean / maximum absolute change.

Condition	T	$p_{\min}$	Model ideal	Support ideal bits/source byte	Quant. ideal bits/source byte	Realized AC	Clipped tokens	$\overline{\Delta s} / \max \Delta s $ bits/token
MinProb	2^{18}	3.81×10^{-6}	4.10	3.79	2.534	2.534	99.0%	$-8.90 / 29.47$
	2^{22}	2.38×10^{-7}	4.10	3.79	3.083	3.083	95.1%	$-5.01 / 25.47$
	2^{26}	1.49×10^{-8}	4.10	3.79	3.539	3.539	62.8%	$-1.77 / 21.47$
	2^{30}	9.31×10^{-10}	4.10	3.79	3.734	3.735	13.8%	$-0.39 / 17.47$
MaxSurprisal/Byte	2^{18}	3.81×10^{-6}	10.53	10.53	9.792	9.790	39.4%	$-1.09 / 16.38$
	2^{22}	2.38×10^{-7}	10.53	10.53	10.138	10.138	19.1%	$-0.58 / 12.38$
	2^{26}	1.49×10^{-8}	10.53	10.53	10.380	10.379	4.2%	$-0.22 / 8.38$
	2^{30}	9.31×10^{-10}	10.53	10.53	10.482	10.481	2.8%	$-0.07 / 4.38$

Tokenizer Fertility. Table 4 provides a natural-text reference point for the fertility metric. While NLP convention often reports tokens per word, our compression analysis uses bytes per token: a larger value means that each model prediction covers more source bytes and can therefore absorb more surprisal before the encoded stream expands. Fertility depends on both the tokenizer and the input, which is why adversarial token sequences can differ substantially from the text8 values reported here.

5.6 Beam Search Experiments

Search Setup. A realized-size beam search approximately maximizes Eq. 5 by cloning the codec state for candidate continuations. Unlike the greedy objectives, it captures arithmetic-coder, metadata, and framing effects during generation. Because the search considers only a bounded beam and candidate set, its result is an approximation to the sequence-level optimum.

Results. [TODO: Report the beam width, branch factor, token and byte budgets, search runtime, and realized-size results; compare against the greedy MinProb and MaxSurprisal/Byte attacks and the conventional-codec baselines.]

Table 3: **Realized size ratio by input and codec.** Values are encoded bytes divided by raw UTF-8 bytes and reported as percentages; lower is better. AC payload excludes the Qwen container and is shown only for cost attribution. All entries in the four codec columns are complete, round-trip-verified streams, and bold marks the smallest complete stream per row.

Input	AC payload	Qwen+AC	Qwen token IDs	FSST	Brotli
text8	14.3%	17.6%	41.3%	56.3%	29.6%
Random printable	94.8%	97.4%	172.1%	98.2%	83.0%
MinProb (v_{tok}^*) adversary	32.9%	36.2%	33.2%	79.5%	46.7%
MaxSurprisal/Byte (v_{byte}^* ; $N = 1,024$)	122.4%	144.3%	154.4%	94.6%	83.2%
One-byte UTF-8 ablation	119.5%	122.8%	225.2%	83.6%	76.1%

Table 4: Tokenizer fertility on the held-out 5 MB text8 test region (855,233 whitespace-delimited words). The text8 BPE is trained only on the first 90 MB with a 32k vocabulary. Fertility is tokens/word (lower is better); bytes/token is the compression-oriented inverse (higher is better). All tokenizers reconstruct the input exactly. The domain-trained BPE produces 4.4% fewer tokens than Qwen.

Tokenizer	Vocab.	Used	Tokens	Tok./word ↓	Bytes/tok. ↑
ASCII byte	128	27	5,000,000	5.8464	1.0000
text8 BPE (32k)	32,000	26,617	933,499	1.0915	5.3562
Qwen2.5	151,665	25,798	976,828	1.1422	5.1186